

Using Large Language Models for BGP Configuration Generation

Dhruv Rauthan

Spring 2025

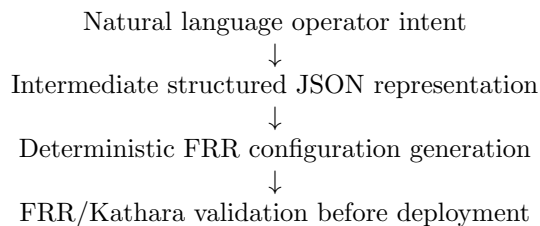
1 Introduction

Border Gateway Protocol (BGP) is traditionally associated with inter-domain routing across the Internet. However, modern data centers and cloud-scale operators such as Meta and Red Hat also use BGP for internal routing, policy control, and dynamic network reachability. This makes BGP configuration a practical systems problem, not just a networking protocol problem. As topologies grow and policies become more customized, even small configuration changes can become time-consuming and error-prone.

The goal of this project was to explore whether Large Language Models (LLMs) can help convert natural language operator intent into syntactically valid and semantically meaningful BGP configurations. More specifically, I focused on FRRouting (FRR) configurations because FRR is open source, widely used in research and emulation workflows, and easy to validate inside container-based network testbeds.

The main motivation came from three constraints that show up in real network operations: cost, privacy, and reliability. Cost matters because using large proprietary models for every network configuration request can quickly become expensive. Privacy matters because network topology, routing policies, peer relationships, and filtering rules are sensitive. Most operators would not want to upload this information to an external service. Reliability matters because a configuration that looks mostly correct can still break routing behavior if a route-map action, sequence number, neighbor direction, or prefix filter is wrong.

For these reasons, the project focused on smaller open-source models such as Llama 3 and DeepSeek variants that can be fine-tuned and run locally. The project evolved from a direct natural-language-to-configuration approach into a more structured pipeline:



The important lesson was that the LLM should not be responsible for every part of the problem. It is useful for understanding the operator's intent, but the final configuration should be generated and checked by deterministic tools wherever possible.

2 Problem Statement

The task can be described as follows: given a human-readable description of a BGP policy, generate the corresponding router configuration. For example, an operator might write:

Create an inbound route-map named INPROV at sequence 10. It should match routes without a valid RPKI record, set community 3:400, and set local preference to 70.

The desired FRR output would be:

```
route-map INPROV permit 10
match rpki notfound
set community 3:400
set local-preference 70
```

This looks simple for one example, but the problem becomes difficult when the input contains multiple route-maps, ambiguous descriptions, missing fields, repeated references such as “this route-map”, and policy attributes that must appear in a specific order. For a recruiter-facing summary, the best way to describe the project is: I built a local LLM-based pipeline that turns natural language BGP policy descriptions into structured configuration objects, converts those objects into FRR syntax, and validates the result using real routing software.

3 Related Work

Existing work on network configuration synthesis usually starts from a more formal input language rather than fully natural language. NetComplete focuses on completing partially written network configurations from sketches and routing requirements. This is useful because operators often modify existing configurations rather than write everything from scratch, but it still requires the user to express intent in a symbolic or structured form.

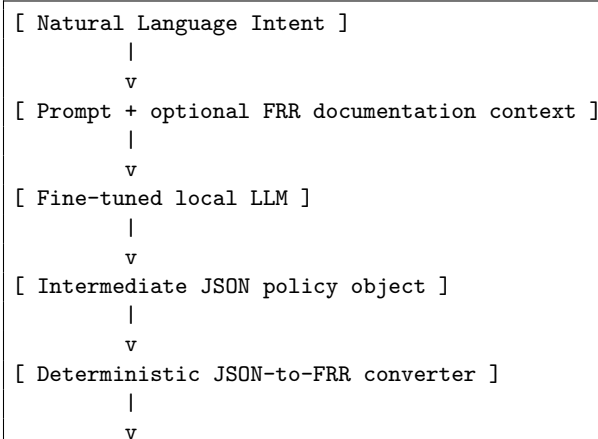
Propane and Propane/AT compile high-level routing policies into BGP configurations using formal specifications, templates, and satisfiability-based reasoning. These systems are strong at correctness and policy reasoning, but they are not designed around a natural language interface or around locally fine-tuned LLMs.

NetConfEval and GENET are closer to this project because they explore translating higher-level intent into configuration-like artifacts. However, their framing is broader and often assumes stronger models or richer topology inputs. In contrast, my focus was narrower and more practical: can smaller local models reliably generate the BGP policy objects that appear in FRR configuration files?

This project is also motivated by industry examples showing that BGP is relevant beyond the public Internet. Meta has described its large-scale BGP-based data center routing design, and Red Hat OpenStack uses FRR for BGP-based dynamic routing in cloud environments.¹² FRR documentation also provides a good target syntax for BGP and route-map generation.³

4 System Overview

The final system design separates intent understanding from configuration rendering and validation. This was a key design decision because smaller LLMs are much more reliable when the output space is constrained.



¹<https://engineering.fb.com/2021/05/13/data-center-engineering/bgp/>

²<https://developers.redhat.com/articles/2022/09/22/learn-about-new-bgp-capabilities-red-hat-openstack-17>

³<https://docs.frrouting.org/en/stable-9.0/bgp.html>

The LLM’s job is to extract the configuration semantics: route-map name, permit or deny action, sequence number, match criteria, and set actions. The deterministic converter’s job is to translate that structured object into FRR commands. The validator’s job is to reject or flag invalid outputs before they are treated as usable configuration.

This architecture is intentionally similar to a compiler. Natural language is the source language, JSON is the intermediate representation, FRR is the target language, and Kathara/FRR act as the runtime validation layer.

5 My Contribution

My main contribution was building and iterating on the end-to-end translation pipeline. I created the synthetic datasets, tested different input/output representations, fine-tuned local models, implemented the JSON-to-FRR conversion logic, and added validation scripts around FRR and Kathara. The project was not just a model experiment. A large part of the work was figuring out where the LLM should be used and where deterministic systems code should take over.

The most important design change I made was moving away from direct CLI generation. Early experiments showed that the model could generate plausible-looking FRR configuration, but plausible was not good enough. A route-map with one missing match clause or an incorrect sequence number can change routing behavior. The intermediate JSON representation made the system easier to validate, easier to debug, and easier to extend.

6 Dataset Construction

Since this is a niche problem, there was no ready-made dataset that directly mapped natural language BGP policy descriptions to FRR configuration. I built a synthetic dataset using FRR examples, BGP policy concepts, vendor documentation, and manually created route-map patterns.

The dataset was built in phases.

6.1 Template-Based Formal Specification

The first dataset used a formal input format with placeholder tokens. A simple example looked like this:

```
router-name: [ROUTER]
as: [AS]
router-id: [ROUTER-ID]
```

The expected output was:

```
hostname [ROUTER]
!
router bgp [AS]
  bgp router-id [ROUTER-ID]
```

This was useful as a first proof of concept, but it was too artificial. The model sometimes copied placeholder values literally instead of replacing them. It also did not solve the real problem because the user still had to speak in a rigid mini-language that was close to the target configuration syntax.

6.2 Realistic Formal Values

The next dataset replaced placeholders with realistic values: router names, AS numbers, IPv4 prefixes, IPv6 prefixes, community strings, route-map names, prefix-list names, and local-preference values. This immediately improved performance because the model no longer had to infer what “[VALUE]” meant. Fine-tuning Llama 3 8B on this version achieved essentially perfect accuracy on a held-out 80/20 split for the formal specification task.

The lesson was that dataset realism mattered more than just dataset size. The model performed better when the input distribution looked like something a network operator might actually write.

6.3 Natural Language to FRR

The next step was to remove the formal input format and pair English descriptions directly with FRR route-map commands. For example:

This inbound route-map named INPROV applies a permit rule at sequence 10. It matches routes that do not have valid RPKI information and then sets the community to 3:400 while adjusting the local preference to 70.

The target output was:

```
route-map INPROV permit 10
match rpki notfound
set community 3:400
set local-preference 70
```

This version was closer to the real use case, but direct generation became fragile as the configurations became longer. Llama 3 8B reached about 90% accuracy on focused route-map examples, and larger models performed better. However, errors remained in places where correctness matters: sequence numbers, omitted match clauses, malformed JSON blocks, and extra natural language around the answer.

6.4 Natural Language to JSON

The final dataset mapped natural language descriptions into a simplified JSON representation. The local project folder contains a route-map dataset with 1,075 examples and a combined multi-route-map dataset with about 1,000 more complex examples. These examples include RPKI validation, communities, prefix lists, AS-path prepending, local preference, metric, weight, route type, origin, and next-hop settings.

The JSON schema looked like this:

```
[
  {
    "routemap_name": null,
    "sequence": null,
    "permit_or_deny": null,
    "direction": null,
    "match": {
      "prefix_list": null,
      "community": null,
      "as_path": null,
      "local_preference": null,
      "origin": null,
      "route_type": null,
      "rpki": null
    },
    "set": {
      "community": null,
      "as_path": null,
      "local_preference": null,
      "origin": null,
      "next_hop": null,
      "metric": null,
      "weight": null
    }
  }
]
```

This schema is inspired by YANG and OpenConfig ideas, but intentionally much simpler. Full YANG models are powerful, but they include deep nesting, vendor-specific extensions, and a large amount of boilerplate. That is not ideal for smaller local models with limited context windows. The simplified JSON format keeps the fields that matter for the current policy generation task while still allowing deterministic conversion into FRR.

7 spaCy Baseline

Before relying on LLMs for the full extraction problem, I explored a traditional NLP baseline using spaCy. The idea was to extract entities and relationships from the natural language description before generating configuration commands.

7.1 Named Entity Recognition

I created custom entity extraction logic for networking objects such as IP addresses, AS numbers, route-map names, prefix-list names, community values, and interface names. Regular expressions worked well for structured tokens because IP addresses and AS numbers follow predictable patterns.

7.2 Dependency Parsing

Dependency parsing was useful for simple attribute-value relationships. For example, in a sentence such as “set local preference to 70”, the parser can help associate the number 70 with the local-preference action. Similarly, route-map names and sequence numbers are often close enough in the sentence to be extracted with a combination of dependency information and rule-based matching.

7.3 Limitations

spaCy worked reasonably well for single-sentence descriptions, but it degraded quickly when the input became more natural. The main problem was context. Users do not always repeat the object name in every sentence. They might say “this route-map”, “the first rule”, or “it should also set community 3:400”. Handling all possible phrasing patterns with hand-written rules became unmanageable.

This was an important failure mode because it pushed the project toward LLM-based extraction. The LLM was better at handling varied sentence structure, but it still needed a constrained output format and validation.

8 Fine-Tuning Setup

The fine-tuning setup used Llama 3 8B with QLoRA-style parameter-efficient fine-tuning. The model was loaded in 4-bit quantized form to fit into available GPU memory, and LoRA adapters were applied to transformer projection layers. The local training script used Unsloth, Hugging Face datasets, and supervised fine-tuning with an Alpaca-style instruction format.

The practical reason for using QLoRA was compute efficiency. Full fine-tuning a larger model would require significantly more GPU memory and longer training time. QLoRA allowed the model to adapt to the BGP configuration domain while keeping the training process feasible on shared cluster resources.

The PACE cluster created a few real constraints during the project. A100, L40, and H100 availability was limited, storage quotas affected model checkpointing, and running containerized validation directly on the cluster was not always smooth because Kathara depends on Docker-like behavior. Because of this, part of the workflow used the cluster for model work and the local machine for Kathara validation.

9 Retrieval-Augmented Generation

Fine-tuning helped the model learn the structure of BGP policy objects, but it did not fully solve syntax reliability. I also explored Retrieval-Augmented Generation (RAG) using FRR documentation, BGP RFC-style references, and syntax examples.

The idea was to retrieve only the documentation chunks relevant to the user's request and include them in the model prompt. For this project, hybrid retrieval made the most sense:

- Dense/vector search helps with semantic matching. For example, “drop routes from an untrusted peer” may relate to route filtering even if the exact command words are not present.
- Sparse/BM25 search helps with exact networking terms such as `route-map`, `prefix-list`, `local-preference`, and `rpki`.
- Reciprocal Rank Fusion can combine both rankings into one context set.

The main reason RAG helps in this problem is that networking configuration is both semantic and syntax-sensitive. The model has to understand the user's intent, but it also has to output exact commands that FRR will accept. A purely semantic retrieval method can miss exact command tokens, while a purely keyword-based method can miss the intent behind the query.

10 JSON to FRR Conversion

Once the model returns JSON, the conversion to FRR is deterministic. For example, the following JSON:

```
{
  "routemap_name": "INPROV",
  "sequence": "10",
  "permit_or_deny": "permit",
  "direction": null,
  "match": {
    "prefix_list": null,
    "community": null,
    "as_path": null,
    "local_preference": null,
    "origin": null,
    "route_type": null,
    "rpki": "notfound"
  },
  "set": {
    "community": "3:400",
    "as_path": null,
    "local_preference": "70",
    "origin": null,
    "next_hop": null,
    "metric": null,
    "weight": null
  }
}
```

becomes:

```
route-map INPROV permit 10
match rpki notfound
set community 3:400
set local-preference 70
```

This separation reduces the risk of malformed syntax. It also makes the system easier to extend. The same JSON could later be translated into Cisco IOS-style configuration, NETCONF XML, or a stricter OpenConfig/YANG representation.

11 Validation with Kathara and FRR

Text similarity is not enough for network configuration. Two configurations can look slightly different but normalize to the same FRR output, while another configuration can look similar but fail to parse. For this reason, I added validation using FRR inside Kathara.

Kathara is a lightweight container-based network emulation framework that can run FRR images.⁴ The validation workflow was:

1. Generate JSON from the natural language prompt.
2. Convert JSON to FRR route-map configuration.
3. Load expected and generated configurations into an FRR container.
4. Use FRR commands to check whether the generated configuration parses.
5. Dump the normalized running configuration with `vttysh`.
6. Compare the normalized generated output against the normalized expected output.

This catches errors that a normal string comparison can miss. It also gives a more realistic signal because the configuration is being interpreted by the same routing suite that would run it.

12 Results

The project should be evaluated across stages rather than as one single number, because each stage tested a different formulation of the problem.

Stage	Observed result	Main takeaway
Placeholder formal specification	Unreliable	The model sometimes copied placeholder values literally and misunderstood the schema.
Realistic formal values	100% on held-out split	Replacing placeholders with realistic values made the task much easier for Llama 3 8B.
Natural language to FRR	About 90% for Llama 3 8B; near-perfect for larger models	Direct CLI generation is feasible for simple cases but fragile for longer policies.
Natural language to JSON	More stable on multi-object prompts	JSON constrained the output and made deterministic conversion possible.
FRR/Kathara validation	Mean normalized daemon similarity of about 0.94 across 88 validation runs	Most generated configs normalized close to expected FRR output, but some failure cases remained.

For the Kathara validation set, 37 of 88 generated configurations produced a perfect normalized daemon match, and 78 of 88 were at or above 0.90 normalized daemon similarity. The average normalized daemon similarity was approximately 0.94. This was more useful than raw text comparison because FRR can normalize or ignore certain formatting differences.

The most common errors were:

- adding a direction field such as `inbound` even when the route-map object itself did not require it;
- confusing `match community` with `set community`;

⁴<https://github.com/KatharaFramework/Kathara>

- partially extracting AS-path prepend values;
- returning extra text around the JSON object;
- adding values that were plausible but not explicitly specified by the prompt.

DeepSeek-style reasoning models were better at returning clean JSON and were strong when all fields were explicitly specified. However, they sometimes inferred missing values too aggressively in novice-level descriptions. Llama 3 was more conservative and often returned `null` when the information was missing, which was preferable for safety. This suggests that model quality should not only be measured by how often it fills fields, but also by how often it refuses to invent missing configuration details.

13 Discussion

The biggest design decision was moving from direct FRR generation to an intermediate JSON representation. Direct generation is attractive because it is simple: the user gives a prompt, and the model returns CLI commands. But that simplicity puts too much responsibility on the model. It has to understand the intent, remember FRR syntax, preserve ordering, avoid extra text, and handle missing information all at the same time.

The JSON representation breaks the problem into smaller pieces. The LLM only extracts structured intent. A deterministic program handles syntax generation. FRR/Kathara handles validation. This is a better division of responsibility for a high-risk domain like network configuration.

Another important tradeoff was simplified JSON versus full YANG/OpenConfig. Full YANG would be more standard and vendor-neutral, but it is too heavy for the initial version of this project. Smaller models struggle when the schema is deeply nested or contains many fields that are irrelevant to the user's immediate intent. The simplified JSON schema is less complete, but it is easier to learn, easier to validate, and easier to translate.

RAG and fine-tuning also solve different problems. Fine-tuning teaches the model the structure of the task. RAG grounds the model in syntax and documentation at inference time. For BGP configuration, the best approach is likely a hybrid: fine-tune the model to produce the JSON schema, then use retrieval to provide exact documentation context for uncommon commands and edge cases.

14 Limitations

The project currently focuses mostly on route-maps and related BGP policy objects. This is a useful starting point because route-maps are common and expressive, but a production system would need broader coverage: neighbors, peer-groups, address families, redistribution, communities, prefix lists, AS-path access lists, RPKI policies, and topology-level constraints.

The current validation is also stronger on syntax than on full network behavior. FRR/Kathara can validate whether configuration parses and normalizes correctly, but functional validation should also include multi-router convergence tests, route propagation checks, ping tests, failover scenarios, and comparisons of routing table snapshots before and after policy changes.

Another limitation is ambiguity in user intent. A human network operator may leave out details that are obvious to them but not obvious to the model. In those cases, the system should not silently guess. A safer version would return the JSON object along with missing fields and ask for confirmation before generating final configuration.

15 Future Work

There are several clear extensions.

- Expand from route-map generation to complete BGP neighbor and address-family configuration.

- Support modification of existing configuration files, which is more practical than generating full files from scratch.
- Add topology-aware validation using multi-router Kathara labs or Batfish.
- Convert the intermediate JSON into NETCONF XML or OpenConfig/YANG for vendor-neutral output.
- Add a human review step where the operator sees the extracted JSON before final FRR generation.
- Compare local fine-tuned models against GPT/Gemini-style proprietary models while measuring cost, latency, privacy, and correctness.

16 Conclusion

This project showed that smaller local LLMs can help generate BGP configurations, but only if the system is designed carefully. Direct natural-language-to-CLI generation works for simple examples, but it becomes unreliable as policies become longer and more ambiguous. The stronger approach is to use the LLM for intent extraction, represent that intent in a simplified JSON schema, deterministically render FRR configuration, and validate the result using real routing software.

From an engineering perspective, the main contribution was not just fine-tuning a model. It was building a safer pipeline around the model: dataset construction, prompt design, structured intermediate representation, deterministic conversion, and validation with FRR/Kathara. That makes the system more practical for the original constraints of the project: local deployment, privacy-preserving inference, lower cost, and higher reliability.